C# اليوم 6 · الأسبوع 2 · ساعتان · دورة أساسيات البرمجة

الحلقات التكرارية والمصفوفات

أهداف التعلم

في نهاية الدرس يستطيع المتدرب أن:

- يكرر الأوامر باستخدام for وwhile.
- يميز متى يستخدم for ومتى يستخدم while.
- يخزن مجموعة قيم في مصفوفة ويصل إليها بالفهرس.
- يمر على المصفوفة بـ foreach ويحسب المجموع والمتوسط والأكبر.

توزيع وقت الحصة

0:00 – 0:10	مراجعة سريعة لدرس الأمس وحل الواجب
0:10 – 0:50	شرح الدرس مع أمثلة حيّة يكتبها المدرب والمتدربون معًا
0:50 – 1:00	استراحة
1:00 – 1:40	تمارين عملية فردية (المدرب يتنقل بين المتدربين الستة)
1:40 – 1:55	تصحيح جماعي ومناقشة الأخطاء الشائعة
1:55 – 2:00	ملخص الدرس + اختبار ذاتي سريع + الواجب

شرح الدرس

1. حلقة for

نستخدمها عندما نعرف عدد مرات التكرار مسبقًا. تتكون من: **البداية**؛ **شرط الاستمرار**؛ **الزيادة**.

C#

```
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine($"المحاولة رقم {i}");
}
```

Output ▸

```
المحاولة رقم 1
المحاولة رقم 2
المحاولة رقم 3
المحاولة رقم 4
المحاولة رقم 5
```

ماذا يفعل هذا الكود؟ برنامج يكرر طباعة سطر واحد خمس مرات باستخدام حلقة for، فيرى الطالب: المحاولة رقم 1 حتى المحاولة رقم 5.

`for (int i = 1; i ⩽ 5; i++)`	الكلمة for تبدأ حلقة تكرار لها ثلاثة أجزاء مفصولة بفاصلة منقوطة: البداية int i = 1 تنشئ عدّادًا قيمته 1، والشرط i <= 5 يعني استمر ما دام العداد أصغر من أو يساوي 5، والخطوة i++ تزيد العداد بمقدار 1 بعد كل دورة.
`{`	الأقواس المعقوصة { و } تجمع مجموعة أسطر في كتلة واحدة، وهنا تحدد الأسطر التي ستتكرر في كل دورة من الحلقة.
`Console.WriteLine($"المحاولة رقم {i}");`	يطبع هذا السطر نصًا ثم ينتقل لسطر جديد. العلامة $ قبل النص تسمح بوضع متغير داخل أقواس { } فتظهر قيمته، لذلك يُطبع رقم الدورة الحالية: 1 ثم 2 ثم ... حتى 5.

2. حلقة while

تستمر ما دام الشرط صحيحًا، وتناسب الحالات التي لا نعرف فيها عدد التكرارات، مثل قوائم البرامج.

C#

```
string password = "";
while (password ≠ "1234")
{
    Console.Write("أدخل كلمة المرور: ");
    password = Console.ReadLine();
}
Console.WriteLine("تم الدخول بنجاح");
```

ماذا يفعل هذا الكود؟ برنامج يطلب كلمة المرور مرارًا حتى يكتب المستخدم 1234، ثم يطبع رسالة نجاح الدخول.

`string password = "";`	ننشئ متغيرًا نصيًا من نوع string اسمه password ونعطيه قيمة فارغة "" في البداية، حتى يكون له قيمة يمكن فحصها في شرط الحلقة.
`while (password ≠ "1234")`	الحلقة while تكرر الكتلة التي بعدها ما دام الشرط صحيحًا. المعامل != يعني «لا يساوي»، أي استمر في التكرار ما دامت كلمة المرور ليست "1234". نستخدم while عندما لا نعرف مسبقًا عدد مرات التكرار.
`{`	الأقواس المعقوصة { و } تجمع الأسطر التي تتكرر داخل الحلقة في كتلة واحدة.
`Console.Write("أدخل كلمة المرور: ");`	الأمر Console.Write يطبع النص دون أن ينتقل إلى سطر جديد، فيكتب المستخدم إجابته بجانب السؤال مباشرة.

`password = Console.ReadLine();`	الأمر Console.ReadLine ينتظر المستخدم حتى يكتب نصًا ويضغط Enter، ثم نخزّن ما كتبه في المتغير password لتُفحص قيمته الجديدة في شرط الحلقة مرة أخرى.
`Console.WriteLine("تم الدخول بنجاح");`	هذا السطر خارج الحلقة، فلا يُنفَّذ إلا بعد أن يصبح الشرط خاطئًا، أي بعد أن يكتب المستخدم 1234 فتنتهي الحلقة.

احذر الحلقة اللانهائية: تأكد أن شيئًا داخل الحلقة يغير الشرط في النهاية.

3. المصفوفات

المصفوفة تخزن عدة قيم من نفس النوع. الفهرس يبدأ من **0**، وعدد العناصر `Length` .

C#

```
int[] grades = { 85, 92, 78, 95, 60, 88 };

Console.WriteLine(grades[0]);       // 85
Console.WriteLine(grades.Length);   // 6

int sum = 0;
int max = grades[0];
foreach (int g in grades)
{
    sum += g;
    if (g > max) max = g;
}
double avg = (double)sum / grades.Length;
Console.WriteLine($"المتوسط: {avg:F1} - الأعلى: {max}");
```

```
Output ▸ 85
6
95 :الأعلى - 83.0 :المتوسط
```

ماذا يفعل هذا الكود؟ برنامج يخزّن ست درجات في مصفوفة، ثم يطبع أول درجة وعدد الدرجات، ويحسب المتوسط وأعلى درجة ويطبعهما.

`int[] grades = { 85, 92, 78, 95, 60, 88 };`	int[] تعني مصفوفة (array) من الأعداد الصحيحة، والمصفوفة متغير واحد يحمل عدة قيم مرتبة. نضع القيم الست بين { } مفصولة بفواصل لتكون القيم الأولى للمصفوفة grades.
`Console.WriteLine(grades[0]); // 85`	نصل إلى عنصر في المصفوفة برقم موقعه بين أقواس مربعة [ ]، ويسمى الفهرس (index). الترقيم يبدأ من 0 لا من 1، لذلك grades[0] هو أول عنصر وقيمته 85. والنص بعد // تعليق يتجاهله الحاسوب.

`Console.WriteLine(grades.Length); // 6`	الخاصية .Length تعطي عدد عناصر المصفوفة، وهنا 6. لاحظ أن آخر فهرس يكون دائمًا Length - 1 أي 5.
`int sum = 0;`	ننشئ متغيرًا sum يبدأ بصفر لنجمع فيه الدرجات واحدة تلو الأخرى.
`int max = grades[0];`	نفترض في البداية أن أعلى درجة هي أول عنصر، ثم سنقارن بقية الدرجات بها ونحدّثها إذا وجدنا درجة أكبر.
`foreach (int g in grades)`	الحلقة foreach تمر على كل عنصر في المصفوفة بالترتيب دون الحاجة إلى عدّاد أو فهرس، وفي كل دورة يحمل المتغير g قيمة العنصر الحالي.
`{`	الأقواس المعقوصة { و } تجمع الأسطر التي تُنفَّذ في كل دورة من الحلقة في كتلة واحدة.
`sum += g;`	المعامل += يعني «أضف إلى القيمة الحالية»، فهذا السطر يساوي sum = sum + g، أي نضيف الدرجة الحالية إلى المجموع.
`if (g > max) max = g;`	إذا كانت الدرجة الحالية g أكبر من أعلى درجة وجدناها حتى الآن، نجعلها هي الأعلى. وعندما يكون بعد if أمر واحد فقط يمكن كتابته دون أقواس { }.
`double avg = (double)sum / grades.Length;`	نحسب المتوسط بقسمة المجموع على عدد الدرجات. الكتابة (double) قبل sum تحوّله إلى عدد عشري (تحويل النوع أو cast)، لأن قسمة عددين صحيحين في C# تحذف الكسور، فنحتاج العشري لنحصل على نتيجة دقيقة.
`Console.WriteLine($"المتوسط: {avg:F1} - الأعلى: {max}");`	نطبع المتوسط وأعلى درجة داخل نص واحد. الكتابة :F1 بعد avg تعني «اعرض الرقم برقم عشري واحد بعد الفاصلة»، فيظهر المتوسط 83.0 بدل رقم طويل.

التمارين العملية

تمرين 1: جدول الضرب

سهل

اقرأ رقمًا واطبع جدول ضربه من 1 إلى 10.

الحل المقترح:

C#

```
Console.Write("الرقم: ");
int n = int.Parse(Console.ReadLine());
for (int i = 1; i ⩽ 10; i++)
    Console.WriteLine($"{n} × {i} = {n * i}");
```

ماذا يفعل هذا الكود؟ برنامج يطلب رقمًا من المستخدم ثم يطبع جدول ضربه من 1 إلى 10، كل عملية في سطر.

`Console.Write("الرقم: ");`	نطبع سؤالًا للمستخدم دون الانتقال إلى سطر جديد، ليكتب الرقم بجانبه.
`int n = int.Parse(Console.ReadLine());`	Console.ReadLine يقرأ ما يكتبه المستخدم كنص، ثم int.Parse يحوّل هذا النص إلى عدد صحيح لنتمكن من الحساب به، ونخزّنه في n. إذا كتب المستخدم شيئًا ليس رقمًا يتوقف البرنامج بخطأ.
`for (int i = 1; i ⩽ 10; i++)`	حلقة for تبدأ العداد i من 1، وتستمر ما دام i <= 10، وتزيده بواحد في كل دورة عبر i++، فتتكرر 10 مرات. لا توجد أقواس { } لأن جسم الحلقة أمر واحد فقط هو السطر التالي.
`Console.WriteLine($"{n} × {i} = {n * i}");`	يطبع سطرًا من جدول الضرب مثل 7 × 3 = 21. داخل { } يمكن وضع عملية حسابية أيضًا، فالتعبير {n * i} يُحسب أولًا ثم تُعرض نتيجته؛ والرمز * هو الضرب في C#، أما × فهو مجرد نص للعرض.

تمرين 2: درجات الطلاب الستة

متوسط

اقرأ درجات 6 طلاب في مصفوفة، ثم اطبع المتوسط وأعلى درجة وأقل درجة وعدد الناجحين (60 فأكثر).

الحل المقترح:

C#

```
int[] g = new int[6];
for (int i = 0; i < g.Length; i++)
{
    Console.Write($"درجة الطالب {i + 1}: ");
    g[i] = int.Parse(Console.ReadLine());
}

int sum = 0, max = g[0], min = g[0], passed = 0;
foreach (int x in g)
{
    sum += x;
    if (x > max) max = x;
    if (x < min) min = x;
    if (x ⩾ 60) passed++;
}
Console.WriteLine($"المتوسط: {(double)sum / g.Length:F1}");
Console.WriteLine($"الأعلى: {max} - الأقل: {min} - الناجحون: {passed}");
```

ماذا يفعل هذا الكود؟ برنامج يطلب درجات ستة طلاب ويخزّنها في مصفوفة، ثم يطبع المتوسط وأعلى درجة وأقل درجة وعدد الناجحين.

`int[] g = new int[6];`	ننشئ مصفوفة أعداد صحيحة اسمها g تتسع لـ6 عناصر. الكلمة new تحجز مكانًا جديدًا في الذاكرة، وتبدأ كل العناصر بقيمة 0 حتى نملأها.

`for (int i = 0; i < g.Length; i++)`	حلقة for يبدأ عدادها من 0 لأن فهارس المصفوفة تبدأ من 0، وتستمر ما دام i < g.Length أي حتى 5، فتمر على المواقع الستة كلها.
`{`	الأقواس المعقوصة { و } تجمع الأسطر التي تتكرر في كل دورة من الحلقة في كتلة واحدة.
`Console.Write($"درجة الطالب {i + 1}: ");`	نطبع سؤالًا يحمل رقم الطالب. نكتب i + 1 لأن العداد يبدأ من 0، والناس يعدّون من 1، فيظهر «درجة الطالب 1» بدل 0.
`g[i] = int.Parse(Console.ReadLine());`	نقرأ ما يكتبه المستخدم ونحوّله إلى عدد صحيح بـ int.Parse، ثم نضعه في الموقع رقم i من المصفوفة.
`int sum = 0, max = g[0], min = g[0], passed = 0;`	نعرّف أربعة متغيرات من النوع int في سطر واحد مفصولة بفواصل: sum للمجموع، وmax وmin يبدآن بأول درجة لنقارن بها، وpassed لعدّ الناجحين.
`foreach (int x in g)`	الحلقة foreach تمر على كل درجة في المصفوفة g بالترتيب، ويحمل المتغير x الدرجة الحالية في كل دورة.
`sum += x;`	المعامل += يضيف x إلى قيمة sum الحالية، أي sum = sum + x.
`if (x > max) max = x;`	إذا كانت الدرجة الحالية أكبر من أعلى درجة حتى الآن، تصبح هي الأعلى الجديدة.
`if (x < min) min = x;`	وبالمثل، إذا كانت الدرجة الحالية أصغر من أقل درجة حتى الآن، تصبح هي الأقل الجديدة.
`if (x ⩾ 60) passed++;`	المعامل >= يعني «أكبر من أو يساوي»؛ فإذا كانت الدرجة 60 أو أكثر نزيد عداد الناجحين بواحد باستخدام ++.
`Console.WriteLine($"المتوسط: {(double)sum / g.Length:F1}");`	نحسب المتوسط مباشرة داخل النص: (double) تحوّل المجموع إلى عدد عشري حتى لا تُحذف الكسور عند القسمة، و:F1 تعرض النتيجة برقم عشري واحد بعد الفاصلة.
`Console.WriteLine($"الأعلى: {max} - الأقل: {min} - الناجحون: {passed}");`	نطبع أعلى درجة وأقل درجة وعدد الناجحين في سطر واحد، كل قيمة داخل { } تُستبدل بقيمة متغيرها.

تمرين 3: تحدٍّ: لعبة تخمين الرقم

تحدٍّ

يختار البرنامج رقمًا عشوائيًا من 1 إلى 100، ويخمّن المستخدم حتى يصيب. بعد كل محاولة اطبع «أكبر» أو «أصغر»، وفي النهاية اطبع عدد المحاولات.

الحل المقترح:

C#

```
int secret = Random.Shared.Next(1, 101);
int guess = 0, tries = 0;

while (guess ≠ secret)
{
    Console.Write("خمّن: ");
    guess = int.Parse(Console.ReadLine());
    tries++;
    if (guess < secret) Console.WriteLine("أكبر ⬆");
    else if (guess > secret) Console.WriteLine("أصغر ⬇");
}
Console.WriteLine($"أحسنت! عرفته في {tries} محاولات");
```

ماذا يفعل هذا الكود؟ لعبة تخمين: يختار البرنامج رقمًا سريًا عشوائيًا من 1 إلى 100، والمستخدم يخمّن ويحصل على تلميح «أكبر» أو «أصغر» حتى يصيب، ثم يرى عدد محاولاته.

`int secret = Random.Shared.Next(1, 101);`	Random.Shared مولّد أرقام عشوائية جاهز في .NET، والدالة Next(1, 101) تعطي عددًا صحيحًا عشوائيًا من 1 حتى 100؛ لأن الحد الثاني 101 (101) غير داخل في النطاق. نخزّن الرقم في secret.
`int guess = 0, tries = 0;`	نعرّف متغيرين: guess لتخمين المستخدم ويبدأ بـ0 (رقم لا يمكن أن يكون هو السر)، وtries لعدّ المحاولات ويبدأ بـ0.
`while (guess ≠ secret)`	الحلقة while تستمر ما دام التخمين لا يساوي (!=) الرقم السري، لأننا لا نعرف مسبقًا كم محاولة سيحتاج المستخدم.
`{`	الأقواس المعقوصة { و } تجمع الأسطر التي تتكرر في كل محاولة في كتلة واحدة.
`Console.Write("خمّن: ");`	نطلب من المستخدم أن يكتب تخمينه بجانب الكلمة دون سطر جديد.
`guess = int.Parse(Console.ReadLine());`	نقرأ ما كتبه المستخدم ونحوّله من نص إلى عدد صحيح بـ int.Parse، ونخزّنه في guess.
`tries++;`	المعامل ++ يزيد قيمة tries بمقدار 1، فنحسب هذه المحاولة.
`if (guess < secret) Console.WriteLine("أكبر ⬆");`	إذا كان التخمين أصغر من الرقم السري، نخبر المستخدم أن الرقم المطلوب أكبر.
`else if (guess > secret) Console.WriteLine("أصغر ⬇");`	else if تعني «وإلا، إذا...»: تُفحص فقط إذا لم يتحقق الشرط السابق؛ فإذا كان التخمين أكبر من السر نخبره أن الرقم أصغر. وإذا كان مساويًا لا يُطبع شيء وتنتهي الحلقة.

`Console.WriteLine($"أحسنت! عرفته في {tries} محاولات");`	بعد الخروج من الحلقة، أي عند التخمين الصحيح، نطبع رسالة تهنئة مع عدد المحاولات المخزّن في tries.

اختبر نفسك (3 أسئلة)

1. كم مرة تُنفذ: for (int i = 0; i < 3; i++)؟

2 | 3 | 4

2. ما فهرس أول عنصر في المصفوفة؟

0 | 1 | 1-

3. أي حلقة تناسب «كرر حتى يكتب المستخدم 0»؟

for | while | لا شيء

الإجابات: 1) 3 2) 2 · 3) 0 · while

الواجب: إعلان المشروع النهائي: اقرأ متطلبات المشروع وابدأ بصفحة HTML الخاصة بفكرتك.